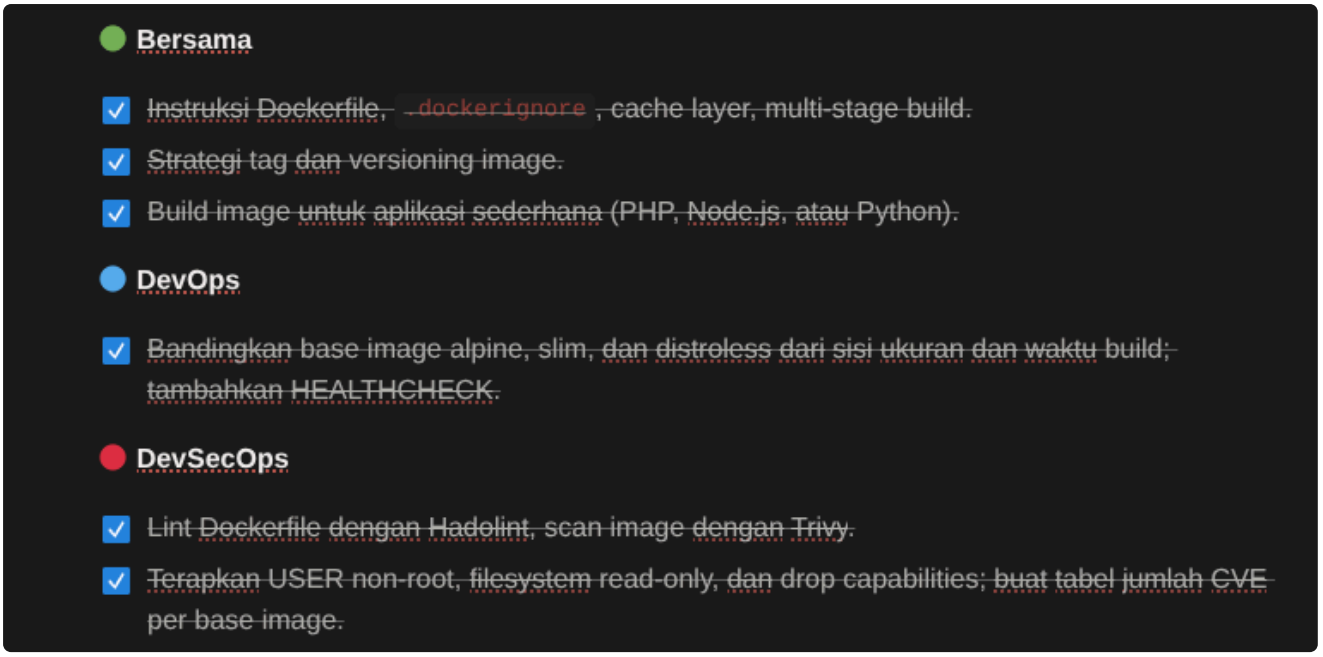
aku akan membuat dokumentasi dari tugas yang ada pada gambar berikut



Dockerfile

nah kita akan langsung mulai dengan checklist pertama itu yang aku awali dengan dockerfile, jadi apa itu dockerfile? intinya dockerfile itu adalah file yang berisi step by step atau instruksi untuk nge build docker image, jadi sebuah dockerfile kalau kita build akan menghasilkan sebuah docker image dan kita akan langsung aja untuk praktek membuat dockerfile sederhana yang mana isinya sebagai berikut:

```
FROM node:20

WORKDIR /app

COPY package.json .

RUN npm install

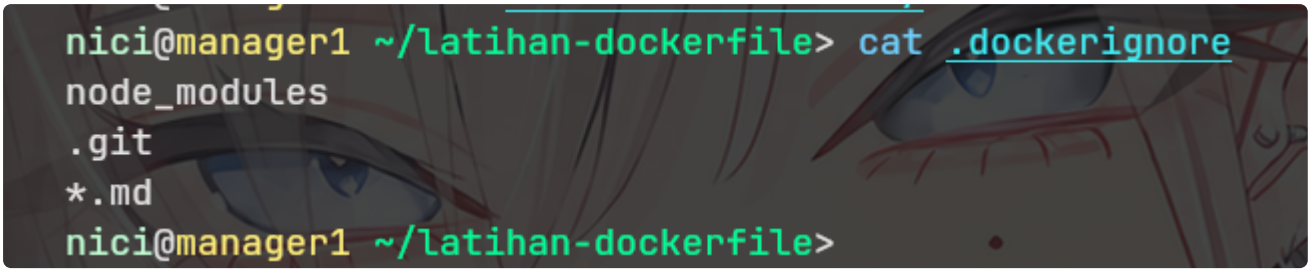
COPY . .

EXPOSE 3000

CMD ["node", "app.js"]
```

kita akan breakdown sedikit soal ini, yang mana FROM node:20 merupakan base image dari container kita, lalu WORKDIR /app yang mana ini adalah direktori kerja yang mana semua instruksi yang ada pada suatu image gitu, jalannya relatif akan disini terus COPY package.json itu tuh kita akan copy file package.json pada direktori dimana dockerfile itu dibuat ke dalam image kita, lalu RUN npm itu kita proses yang akan dieksekusi waktu

proses build container yang mana saat sudah ter build, maka modul modul js yang digunakan akan terinstall otomatis jadi kita gausah install ilang lagi modul modulnya, lalu COPY .. yang mana ini kita copy semua file file lain yang ada pada direktori proyek kaya file web dll juga termasuk, terus EXPOSE 3000 itu si container akan listen di port 3000, bukan expose ke host yang mana kalo nanti mau expose ke host ya pake docker run -p 80:3000 yang maksudnya host akan buka port ke port 3000 nya container jadi saat kita akses ke port 80 nya host itu akan disambungkan ke port 3000 nya container, lalu CMD ["node", "app.js"] itu command yang dijalanin saat container itu start, ya kira kira mirip dengan init lah ya, dan dockerfile hanya boleh ada satu CMD dan kalau ada lebih sari satu maka yang terakhir itu yang akan digunakan, dan lanjut ke .dockerignore yang mana ini kurang lebih kaya .gitignore yang mana akan aku contohkan isi dari .dockerignore ku yang sebagai berikut:



nah maksudnya .dockerignore itu adalah kita itu menyaring file atau folder mana yang perlu untuk disaring agar tidak ter copy saat proses build yang mana kita wajib menyaring file file seperti .git, .env atau file yang sejenisnya karena kalau udah tercopy pas waktu proses COPY .. ya udah wassalam kalau kita up containernya,lalu ada cache layer yang mana intinyaa ini tuh cache yang ada pada proses build dan contohnya saat kita ngebuild suatu dockerfile yang berisi 5 barit perintah dan waktu kita mau build ulang dan hasilnya kira kira bakalan sama kaya build sebelumnya ya docker gaakan ngebuild ulang, hanya pake hasil yang udah aja cukup tapi intinya kalo ada satu urutan dockerfile yang berubah maka hasil sebelumnya akan langsung invalid karena kan hasilnya juga bener bener beda jadinya kalau mau nulis dockerfile itu kita wajib banget buat pakai urutan yang sama biar ga dikit dikit compile ulang, dikit dikit compile ulang, lalu ada multi staged build yang mana contohnya seperti ini,

Multi-stage Build

```
FROM node:20 AS builder

WORKDIR /app

COPY package.json .

RUN npm install

COPY . .

FROM node:20-slim
```

```
WORKDIR /app

COPY --from=builder /app .

EXPOSE 3000

CMD ["node", "app.js"]
```

nah disitu kita punya 2 build dalam satu dockerfile yang mana kalau kita sedikit cermati pada build kedua, maka disitu kita bukannya menggunakan berbagai tools build baru pada satu dockerfile yang kalau gitu akan menghasilkan docker image yang besar dan disini kita menggunakan hasil yang udah jadi aja ke build selanjutnya, simpelnya kaya kita pakai satu variabel pada suatu fungsi yang mana itu bisa juga digunakan pada fungsi selanjutnya, oh ya lalu kita juga akan masuk ke strategi tag dan versioning image, yah disini kita hanya fokus ke penamaan hasil build aja sih yang intinya banget kaya gini,

Tag dan Versioning Image

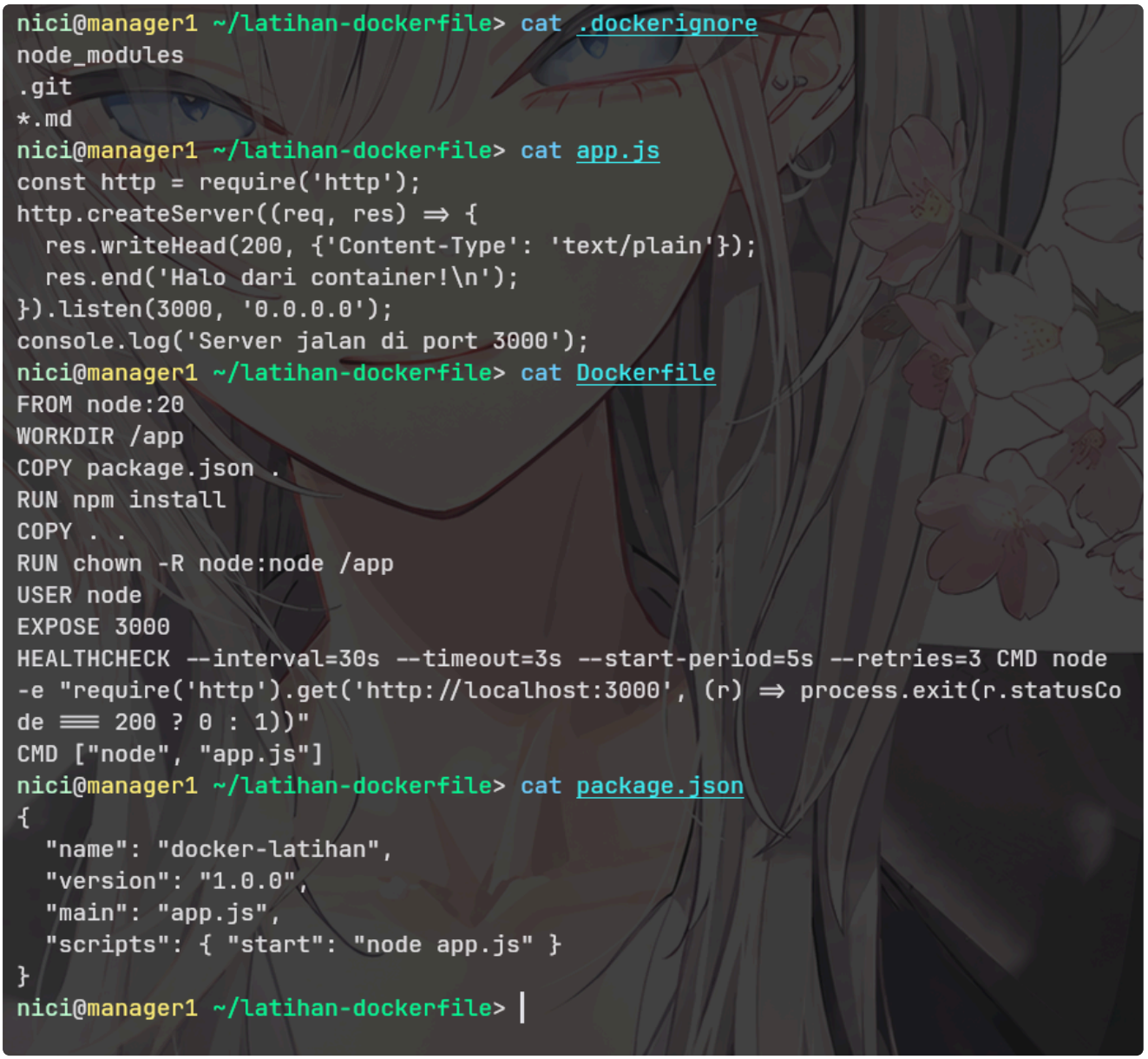
```
docker build -t latihan-app:1.0.0 .

docker build -t latihan-app:1.0.1 .   # bug fix kecil

docker build -t latihan-app:1.1.0 .   # fitur baru

docker build -t latihan-app:2.0.0 .   # breaking change
```

gitu doang, lalu kita akan coba build image untuk aplikasi sederhana yang mana kita akan membuat app.js yang isinya cuma "server jalan di port 3000", gitu doang dan kita akan langsung membuat dockerfile, dockerignore, dan package.json, dan app,js nya sendiri yang mana isi dari masing masing file tersebut kira kira adalah

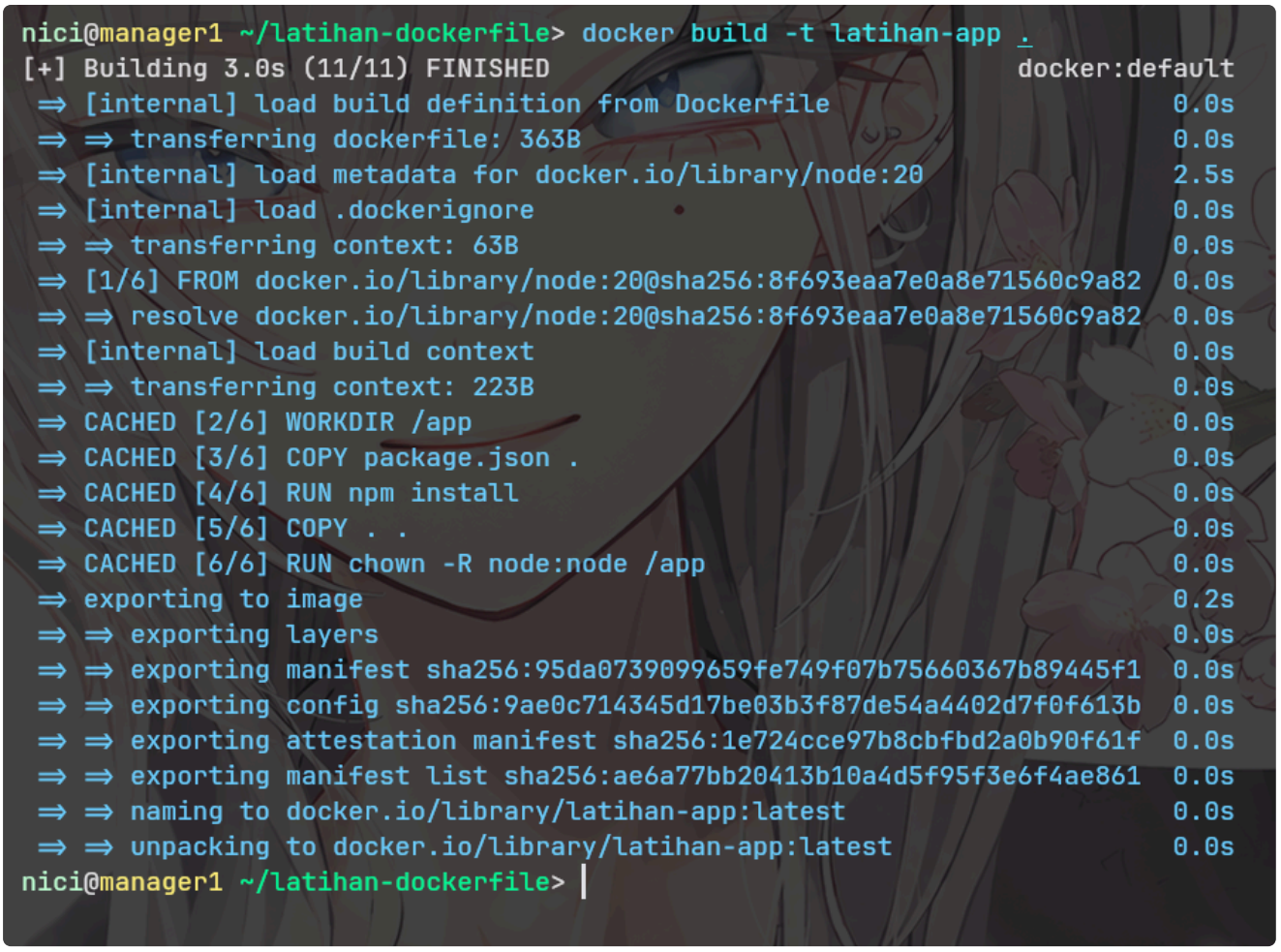


Build Image

pertama kita build dulu dengan command

```
docker build -t latihan-app .
```

lalu hasilnya adalah



Menjalankan Container

abis itu kita jalankan dengan command

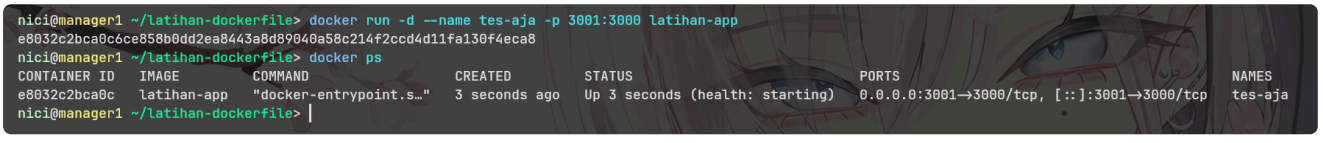
```
docker run -d --name tes-aja -p 3001:3000 latihan-app
```

yang man aitu kita menjalankan image dengan nama tes aja lalu port host yang akan kita buka itu port berapa dan image yang akan kita gunakan itu apa dan kita akan cek dengan command

Cek Container

```
docker ps
```

yang mana hasilnya kira kira seperti ini,



Test Container

setelah itu kita akan tes dengan command

```
curl localhost:3001
```

kenapa 3001? karena host kita open di 3001 jadi hasilnya adalah

```
nici@manager1 ~/latihan-dockerfile> curl localhost:3001
Halo dari container!
nici@manager1 ~/latihan-dockerfile> cat app.js
const http = require('http');
http.createServer((req, res) => {
  res.writeHead(200, {'Content-Type': 'text/plain'});
  res.end('Halo dari container!\n');
}).listen(3000, '0.0.0.0');
console.log('Server jalan di port 3000');
nici@manager1 ~/latihan-dockerfile>
```

Perbandingan Base Image

Bandingin base image alpine, slim, distroless + HEALTHCHECK

abis itu kita akan bandingin base image yang beda beda

FULL (node:20) yang intinya lengkap isinya, banyak tools, paling gede tapi paling gampang di-debug

ALPINE (node:20-alpine) yang intinya berbasis alpine linux, pake musl libc (bukan glibc), jauh lebih kecil, tapi kadang ada masalah compatibility library tertentu. aku belum tau untuk penggunaan kaya gini tuh kaya gmn tapi pas aku pake void linux dengan library musl tu rasanya bikin nyawa berkurang

SLIM (node:20-slim) ini berbasis debian tapi ya ga sekecil apline cuma ya tetap dikurang kurangi, ga sekecil alpine tapi tetep pake glibc jadi lebih compatible, intinya agak tengah tengah

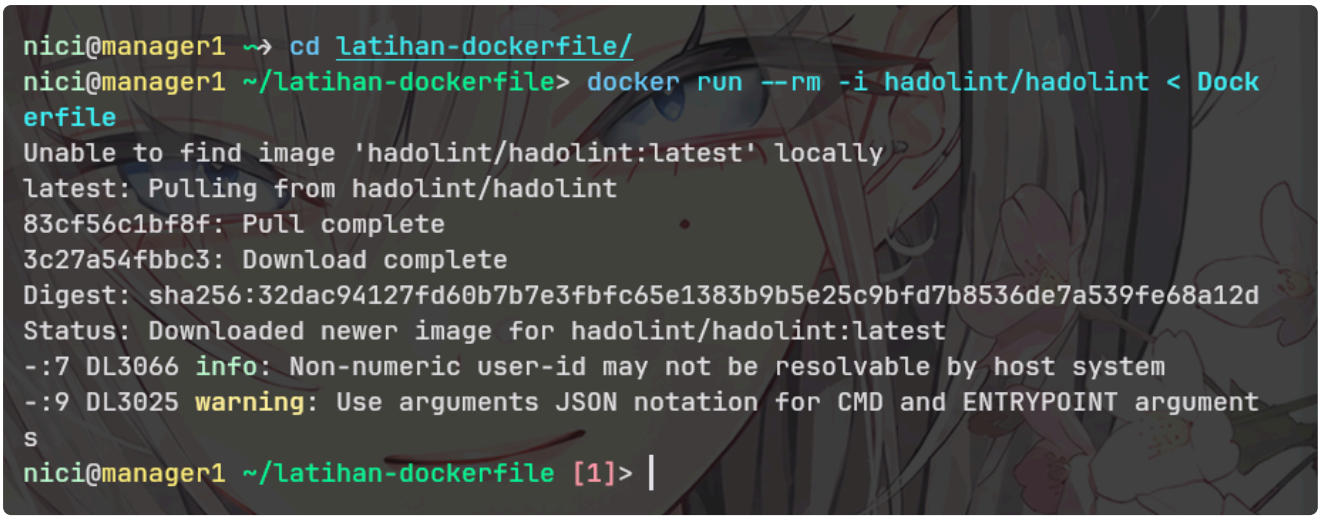
DISTROLESS yang intinya bahkan ga ada shell sama sekali di dalemnya, cuma runtime sama app doang, paling kecil dan paling aman (attack surface minimal), tapi paling susah di debug soalnya ya ga bisa docker exec masuk shell

Hadolint

langsung aja kita akan coba lint dockerfile dengan hadolint, yang mana intinya lint itu kita ngecek apakah ada suatu kesalahan atau yang sejenisnya pada dockerfile sebelum file tersebut digunakan yang mana aku pribadi nyebutnya dengan sebutan "ngelinting" lalu langsung aja kita akan praktek dengan dockerfile yang udah kita buat sebelumnya dengan langsung jalanin hadolint via docker untuk langsung mengecek dockerfile kita dengan command

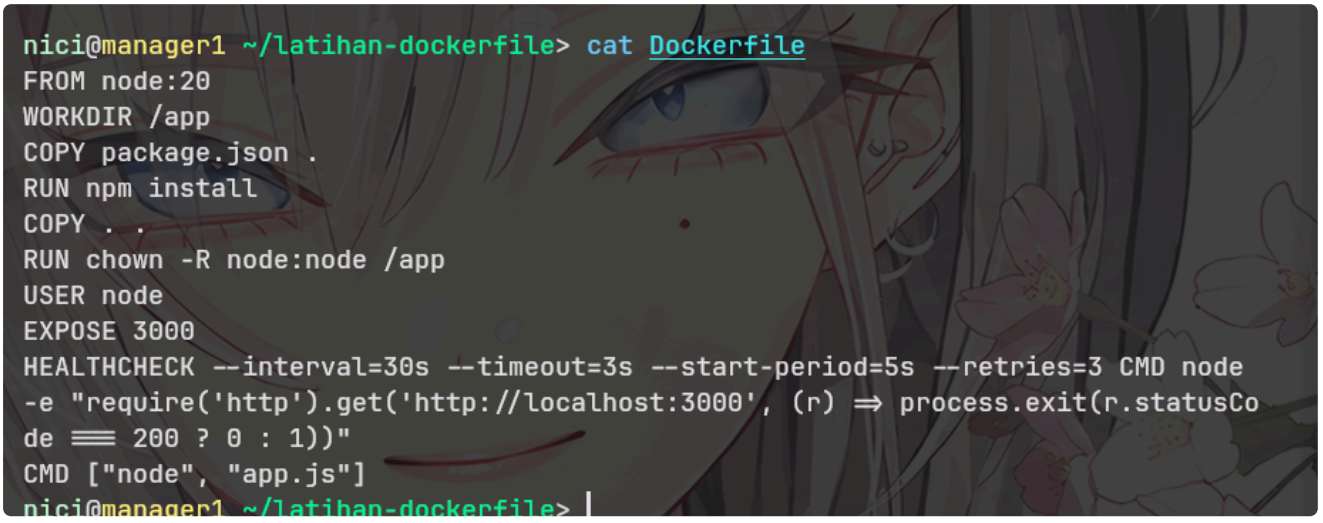
```
docker run --rm -i hadolint/hadolint < Dockerfile
```

lalu hasilnya kira kira seperti ini

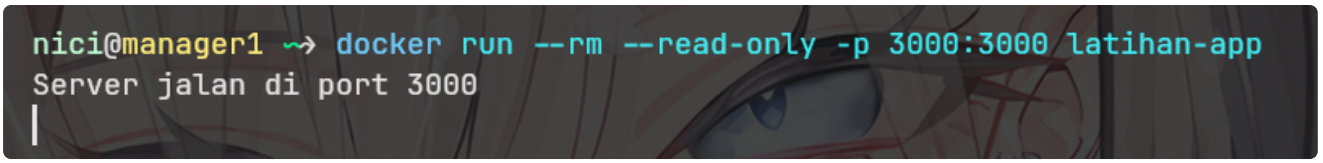


User Non-Root

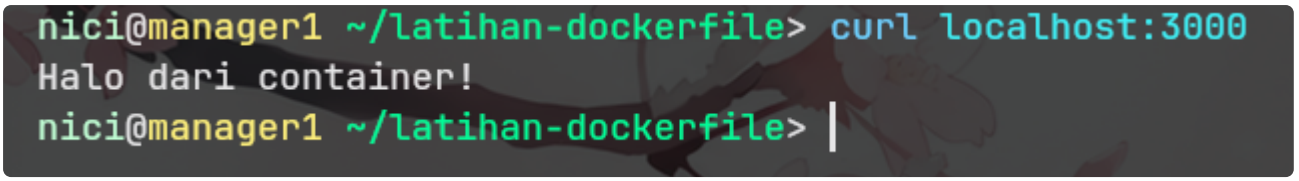
lalu kita akan menerapkan user non root karena by default, container jika kita tidak teliti akan membuat kita menjalankan itu dengan user root dan ini yang akan kita berantas yang mana dari awal itu dockerfile kita sudah menerapkan untuk penggunaan user non root, yang mana jika kita lihat lagi tuh isi dari dockerfile kita kira kira seperti ini,



pada bagian user itu aku menulis USER node yang mana artinya disitu kita menggunakan user node yang mana itu bukanlah root, kenapa harus tidak root? ya kalu app kita kena exploit dan disitu container nya tidak menggunakan user root, maka tidak banyak yang bisa dilakukan oleh penyerang dan kalau root bagaimana? ya wassalam, gitu deh dan kita akan lanjut ke bagian penggunaan read only filesystem yang mana ini intinya kita akan membuat si container itu bersifat read only pada filesystem nya jadi kalau ada hacker masuk dan itupun juga user nya bukan root ya, jadi dia udah gabisa akses sana sini dan juga dia juga mungkin tidak bisa melakukan operasi write pada filesystem kita jadi dapat meminimalisir kerusakan aja sih dan cara melakukannya pun juga cukup mudah dengan nambahin --read-only aja pada command nya, contohnya ini,



nah disitu tertera bahwa container sudah berjalan dan itu jalan di port 3000 dan saat kira coba curl akan menghasilkan output,

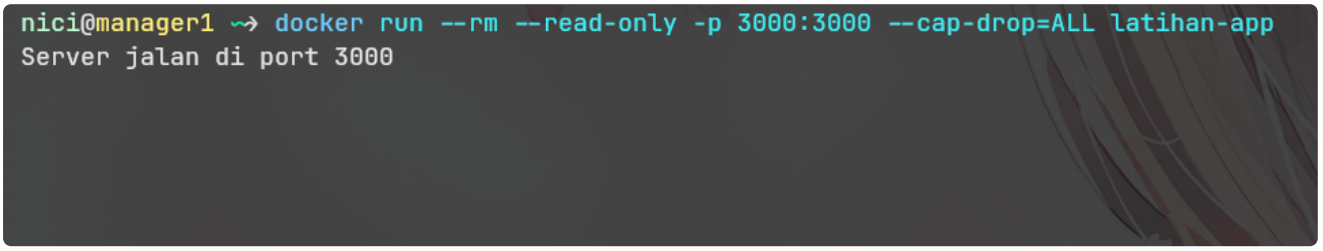


Drop Capabilities

cukup untuk read only dan lanjut ke drop capabilities yang mana intinya kita ngurangin linux previlegge yang ada pada proses di dalam container, jadi selama container tuh kita cuma perlu pake apa yang diperlukan sana dan yang tidak diperlukan kita drop, sebenernya capabilities yang bisa digunakan juga cukup banyak pada container hanya dengan --cap-drop=all itu sudah akan membuang semua capabilities tambahan yang tidak perlu untuk digunakan karena kalau container secara default itu akan menggunakan semua capability jadi disini kita ingin membuang dan hanya menggunakan yang diperlukan saja dan langsung aku akan pakai command

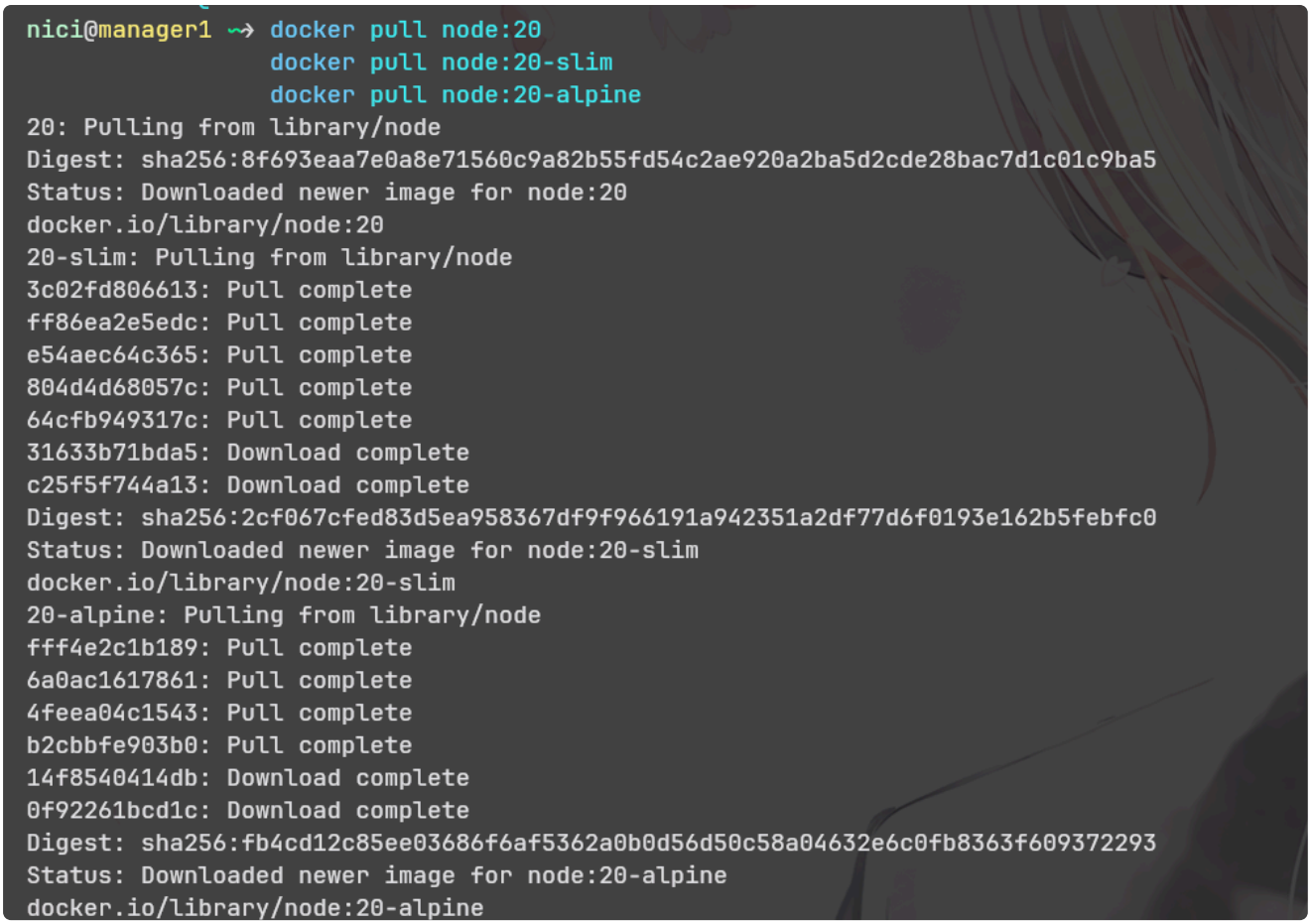
```
docker run --rm --read-only -p 3000:3000 --cap-drop=ALL latihan-app
```

yang hasilnya kira kira seperti ini,



Trivy

lalu kita akn lanjut ke pembuatan tabel yang mana pertama kita akan menggunakan trivy untuk mengecek keamanan dari suatu image dan disini kita akan menggunakan 3 image yang mana itu adalah node:20 versi biasa, slim, dan alpine yang mana kita akan pull ketiganya dulu dengan command dan output seperti berikut,



dan langsung aja kita akan menggunakan trivy untuk ketiga image tersebut dengan command

```
docker run --rm

-v /var/run/docker.sock:/var/run/docker.sock

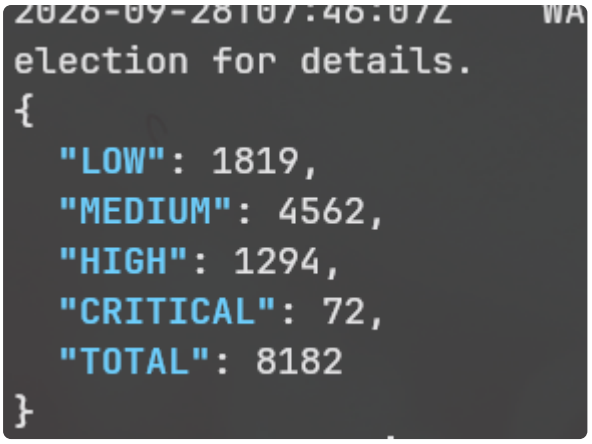
aquasec/trivy\:latest image --quiet node:20 2>/dev/null | tail -n 5

docker run --rm

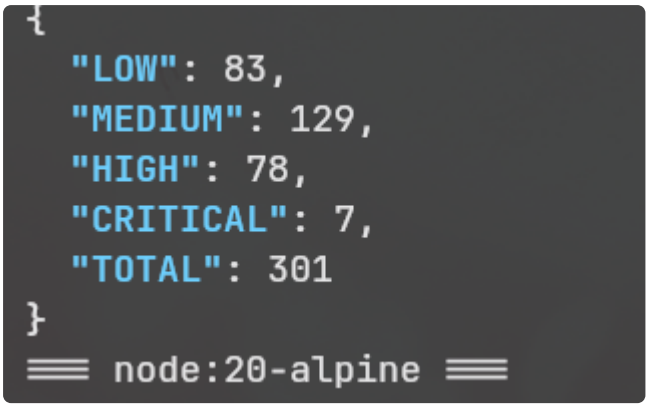
-v /var/run/docker.sock:/var/run/docker.sock

aquasec/trivy\:latest image node:20
```

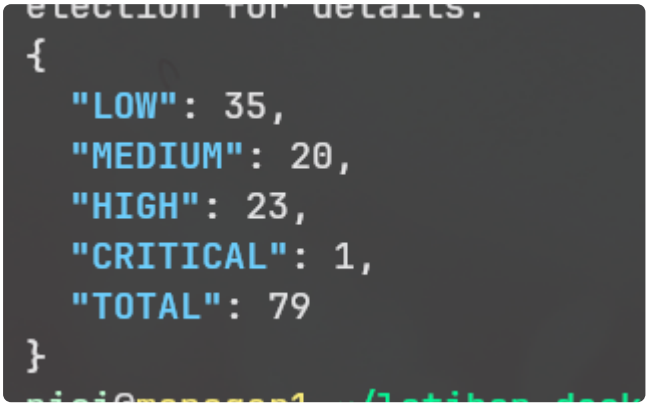
dan kita akan melakukan hal yang sama untuk image kedua dan ketiga lalu outputnya kira kira seperti ini untuk yang node:20



untuk yang versi slim seperti ini



dan yang alpine seperti ini,



Hasil Scan CVE

lalu jika kita buat tabelnya hasilnya kira kira seperti ini,

Base Image LOW MEDIUM HIGH CRITICAL Total CVE					
`node:20`	1.819	4.562	1.294	72	**8.182**
`node:20-slim`	83	129	78	7	**301**
`node:20-alpine`	35	20	23	1	**79**

segitu saja kalo dari aku, makasihh